

# SIMATS ENGINEERING

## ASSESSMENT TOOL 3

**COURSE CODE: CSA14**

**COURSE NAME: Compiler Design**

---

### COURSE OUTCOME COVERED

**CO2:** Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)

*Assessment Tool Weightage: 20%*

---

**QUESTIONS: 20**

**Total Marks:50**

Annexure A

### VIVA VOCE

---

#### *Code of Conduct:*

*I K.Madhulika(192524190) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student: K.Madhulika**

| Criteria                  | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---------------------------|-----------|---------------------|----------------------|----------------------|---------------------|
| Understanding of Concepts | 25%       |                     |                      |                      |                     |
| Explanation               | 25%       |                     |                      |                      |                     |
| Application               | 20%       |                     |                      |                      |                     |
| Organization              | 15%       |                     |                      |                      |                     |
| Accuracy                  | 10%       |                     |                      |                      |                     |
| Clarity                   | 5%        |                     |                      |                      |                     |

Faculty Incharge

## Annexure A

### SHORT ANSWER TEST

1. How would you distinguish whether an error belongs to lexical analysis or syntax analysis?
2. How would the design of a compiler change if syntax analysis were completely removed?
3. Why does the parser require grammar information instead of simply checking whether tokens are present?
4. If the lexical analyzer produces a valid sequence of tokens but the program is rejected by the parser, what could be the possible reasons?
5. Why is syntax analysis necessary even after lexical analysis has successfully generated tokens?
6. How would you prove that a grammar is ambiguous?
7. If a parser accepts an invalid program, which component of the compiler might be responsible, and why?
8. If two different programs generate the same sequence of tokens, how can the parser distinguish their structures?
9. How can a parse tree help identify the source of a syntax error?
10. Why is an AST preferred over a complete parse tree during later compiler phases?
11. How would you design a parser to provide meaningful error messages instead of simply reporting "Syntax Error"?
12. Given a grammar, how would you determine whether it can generate a particular input string?
13. If a grammar generates two different parse trees for the same string, what problem can occur during code generation?
14. How would ambiguity in an expression grammar affect the meaning of an expression?
15. Why is the grammar  $E \rightarrow E + E | E * E | id$  ambiguous?
16. How would you modify an ambiguous expression grammar to enforce operator precedence?
17. How would you modify a grammar to enforce left or right associativity?
18. If a grammar contains left recursion, which parsing techniques may fail and why?
19. How would you eliminate immediate left recursion from a grammar?
20. How would you eliminate indirect left recursion from a grammar?
21. Why is left factoring necessary for some top-down parsing techniques?
22. How would you transform a grammar having common prefixes to make it suitable for predictive parsing?
23. Can two different CFGs generate the same language? Justify your answer.
24. How would you determine whether a grammar is suitable for LL(1) parsing?
25. If a grammar is ambiguous, can it be directly used for deterministic parsing? Why or why not?
26. Why does top-down parsing naturally correspond to leftmost derivation?
27. How would the parsing process change if rightmost derivation were used in a top-down parser?
28. Why does top-down parsing have difficulty handling left-recursive grammars?
29. How would you identify whether a grammar will cause infinite recursion in a recursive descent parser?
30. When would you prefer top-down parsing over bottom-up parsing?
31. Why can a grammar accepted by an LR parser fail to be accepted by an LL(1) parser?
32. How would you compare the efficiency of a backtracking parser and a predictive parser?
33. If a grammar requires extensive backtracking, what modification could improve its performance?
34. How does grammar transformation affect the structure of the resulting parse tree?
35. Can removing left recursion change the associativity of operators? Explain.
36. Why is grammar transformation sometimes necessary even when the original grammar is logically correct?
37. How would you choose between recursive descent and table-driven predictive parsing for a compiler?
38. What kind of grammar structure makes top-down parsing difficult?
39. How can lookahead reduce the need for backtracking?

40. What would happen if the parser had insufficient lookahead to choose a production?
41. Given a grammar, how would you convert each production into recursive descent procedures?
42. How would you identify a recursive descent parser that may enter infinite recursion?
43. Why does direct left recursion cause infinite recursive calls?
44. How would you modify a recursive descent parser to handle an expression grammar correctly?
45. If two productions begin with the same terminal, how can a recursive descent parser decide between them?
46. What is the effect of backtracking on the time complexity of recursive descent parsing?
47. How would you improve a recursive descent parser that repeatedly tries unsuccessful alternatives?
48. Why is recursive descent parsing considered easier to debug than some table-driven parsers?
49. How would you handle syntax errors in a recursive descent parser?
50. If a recursive descent parser accepts an incomplete input string, what could be wrong with its implementation?
51. How would you construct an LL(1) parsing table for a given grammar?
52. How can a conflict in a predictive parsing table indicate a problem with the grammar?
53. If two productions are entered into the same parsing-table cell, what does this indicate?
54. How would you use FIRST and FOLLOW sets to determine whether a grammar is LL(1)?
55. Why is FOLLOW required when a production can derive  $\epsilon$ ?
56. How would you calculate FIRST for a string containing multiple grammar symbols?
57. How would you calculate FOLLOW for the start symbol?
58. What happens to FIRST and FOLLOW sets if a new  $\epsilon$ -production is added?
59. How can an incorrect FOLLOW set lead to incorrect parsing-table entries?
60. How would you identify an LL(1) conflict caused by common prefixes?
61. How would you remove an LL(1) conflict using left factoring?
62. How would you determine whether a grammar requires more than one lookahead symbol?
63. Can a grammar be unambiguous but not LL(1)? Justify your answer.
64. Can a grammar be LL(1) but contain unnecessary productions? Explain.
65. How would you test an LL(1) parser using an invalid input string?
66. Why does bottom-up parsing use rightmost derivation in reverse?
67. How would you identify the handle of a sentential form?
68. Why is selecting the correct handle essential in bottom-up parsing?
69. What could happen if a parser reduces the wrong substring?
70. How does a shift-reduce parser decide whether to shift or reduce?
71. How would you trace the stack and input buffer during shift-reduce parsing?
72. Why can a shift-reduce conflict occur in an ambiguous grammar?
73. How would operator precedence help resolve a shift-reduce conflict?
74. How would you distinguish a shift-reduce conflict from a reduce-reduce conflict?
75. Which conflict is generally more difficult to resolve, and why?
76. If two reductions are possible at the same point, how can the parser decide which one to use?
77. How would you identify whether a grammar is suitable for shift-reduce parsing?
78. Why are handles not always immediately visible in the input string?
79. How does bottom-up parsing construct the parse tree differently from top-down parsing?
80. If a shift-reduce parser repeatedly shifts without reducing, what could be the reason?
81. How would you construct an operator precedence table for an expression grammar?
82. How does operator precedence resolve ambiguity in arithmetic expressions?
83. How would you determine whether multiplication or addition should be reduced first?
84. How does associativity affect operator precedence parsing?
85. Why is an operator precedence parser unsuitable for grammars containing adjacent non-terminals?
86. Why can operator precedence parsing not directly handle  $\epsilon$ -productions?
87. How would you detect a conflict in an operator precedence table?
88. If two operators have the same precedence, how would associativity determine the parsing action?
89. How would you modify an expression grammar to support a new operator?
90. How would you analyze the expression  $a + b * c$  using operator precedence?
91. Why are LR parsers more powerful than LL(1) parsers?
92. How would you construct LR(0) items for a given grammar?
93. What is the purpose of the dot in an LR item?
94. How would you determine whether an LR state requires a shift or reduce action?

95. Why is an augmented grammar used in LR parsing?
96. How does the SLR parser use FOLLOW sets during reduction?
97. Why can an LR(0) grammar fail to be SLR(1)?
98. How would you identify a shift-reduce conflict in an SLR parsing table?
99. How would you compare SLR, CLR, and LALR based on power and table size?
100. If a grammar produces conflicts in an SLR parser, how would you decide whether to modify the grammar or use a more powerful parser?

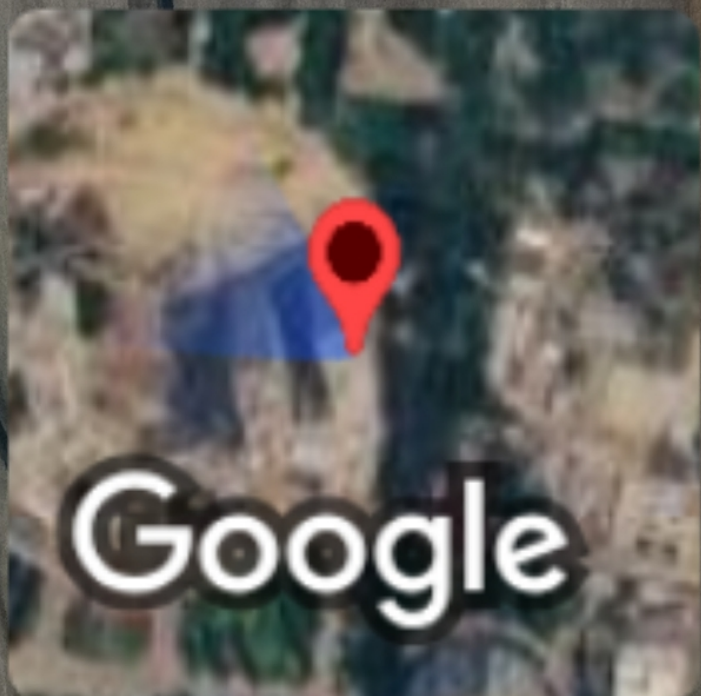
## RUBRICS FOR CALCULATION

| Criteria                 | Weightage | Level 4<br>(Exemplary)                                                    | Level 3 (Proficient)                                 | Level 2<br>(Developing)                              | Level 1<br>(Beginning)                     |
|--------------------------|-----------|---------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------------|--------------------------------------------|
| Conceptual Understanding | 25        | Demonstrates deep understanding of concepts with complete clarity         | Explains concepts correctly with minor gaps          | Partial understanding with errors or omissions       | Unable to explain basic concepts           |
| Application              | 25        | Effectively applies concepts to new situations; solves problems logically | Applies concepts with minor errors                   | Limited ability to apply concepts; requires guidance | Unable to apply concepts effectively       |
| Analytical Thinking      | 20        | Critically analyzes scenarios with strong justification and reasoning     | Provides reasonable analysis with some justification | Superficial analysis with weak reasoning             | No analytical reasoning demonstrated       |
| Communication            | 15        | Communicates ideas clearly, confidently, and with appropriate terminology | Communicates clearly but with minor hesitation       | Communication lacks clarity or structure             | Unable to communicate ideas effectively    |
| Response to Questions    | 15        | Responds accurately and confidently, extending answers with depth         | Responds correctly but with limited depth            | Responds partially; requires prompting               | Unable to respond to questions effectively |





 GPS Map Camera



Mevalurkuppam, Tamil Nadu,  
India 

22g8+cwh, Mevalurkuppam, Tamil Nadu 602105,  
India

Lat 13.026453° Long 80.01722°

Monday, 03/08/2026 10:57 AM GMT +05:30